PHP provides a useful built-in function, `number_format()`, to make life easier. This function requires a minimum of one argument - the number (as a float) to be formatted - and up to three optional arguments to control the output: the number of decimal places, the character to use as the decimal separator, and the character to use as the thousands separator:

```
$number = 100000000000.01;

// number_format(
//     float $num,
//     int $decimals = 0,
//     ?string $decimal_separator = ".",
//     ?string $thousands_separator = ","
// ): string
print number_format($number, 0, '', '') . PHP_EOL;
print number_format($number, 2, '.', ',') . PHP_EOL;
print number_format($number, 2, ',', ' ') . PHP_EOL;

Output:
100000000000
100,000,000,000.01
100 000 000 000,01
```

Here the number *100000000000.01* is formatted to different locales (the French convention is to separate thousands with a space and use a comma as the decimal separator).

Operators, Constants, and Built-In Functions

We've already covered operators and their behavior in a previous chapter, but let's recap the ones that are used for numeric operations. The language operators of PHP that are relevant to mathematics are fairly intuitive. `$a + $b` will add two variables together either as integers (if both are integers) or floats (if either are floats), casting as necessary (if possible). `$a * $b` will multiply the operands, and so on.